Vấn Đề: Exploding Gradients (Bùng nổ Gradient)

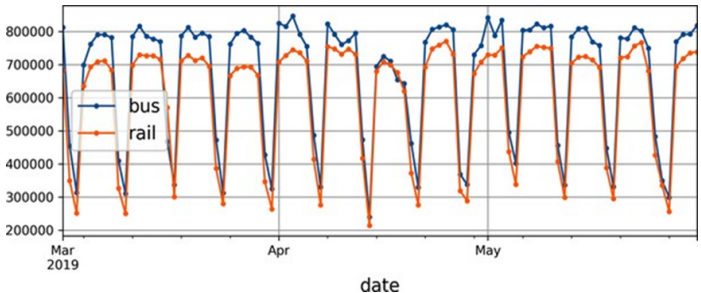
- Ngược lại, nếu giá trị riêng lớn nhất của $W_{hh} > 1$, gradient bị nhân lên theo cấp số nhân và phát nổ thành *NaN* hoặc vô cực.
- **Hậu quả:** Cập nhật trọng số quá lớn, làm mất ổn định hoàn toàn mạng, Loss dao động điên cuồng.
- **Giải pháp cho bùng nổ:**
  - **Gradient Clipping:** Cắt cụt gradient nếu nó vượt quá một ngưỡng nhất định trước khi cập nhật.
  - Thường xuyên được áp dụng mặc định trong các Framework như Keras/PyTorch.

Bài toán Dự báo Chuỗi Thời Gian Thực Tế

- Chúng ta sẽ xem xét dữ liệu thực tế để hiểu tại sao cần RNN phức tạp.
- Dự báo chuỗi thời gian (Time series forecasting) thường phải xử lý:
  - Nhiễu trắng (White noise) ngẫu nhiên.
  - Tính thời vụ (Seasonality) - Các mẫu lặp lại đặn (Ngày, tuần, tháng).
  - Xu hướng (Trend) - Tăng trưởng hoặc suy giảm chậm trong thời gian dài.
- Tiền xử lý dữ liệu để loại bỏ những yếu tố này đôi khi hiệu quả hơn để mạng nơ-ron tự học từ đầu.

Ví dụ: Lượng hành khách xe buýt (Chicago)

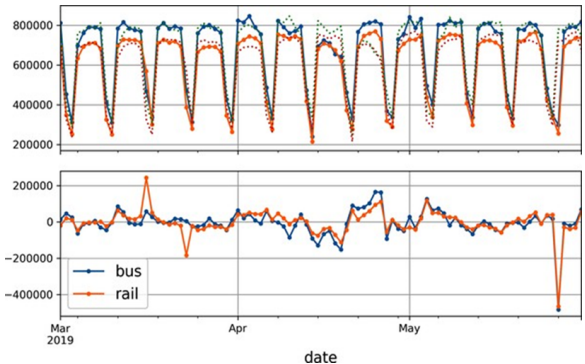
- Tập dữ liệu lượng hành khách hàng ngày ở Chicago (2019).
- Trục hoành là Ngày, trục tung là số hành khách.
- **Hình 15-6** cho thấy các biến động liên tục và nhịp nhàng. Mỗi "khối"lớm chớm tương ứng với một tuần làm việc (cao điểm) và cuối tuần (giảm sút).



Hình 15-6: Lượng hành khách hằng ngày ở Chicago

Kỹ thuật Tiền Xử Lý: Sai phân (Differencing)

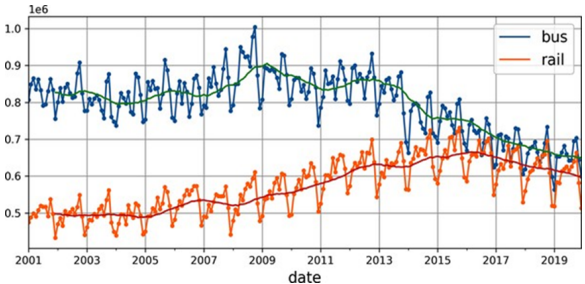
- Để giúp mạng dễ dàng học hơn, ta có thể khử **tính thời vụ hàng tuần** bằng phép tính sai phân trễ 7 ngày ($y_t - y_{t-7}$).
- **Hình 15-7** cho thấy: Biểu đồ trên mô tả chuỗi gốc so với chuỗi bị dịch (trễ).
- Biểu đồ dưới (chuỗi sai phân) mượt mà, stationary (dừng) và loại bỏ được nhiễu do khác biệt giữa ngày thường và cuối tuần.



Hình 15-7: Chuỗi thời gian trễ 7 ngày và phép sai phân

Kỹ thuật Tiền Xử Lý: Xu Hướng Vĩ Mô

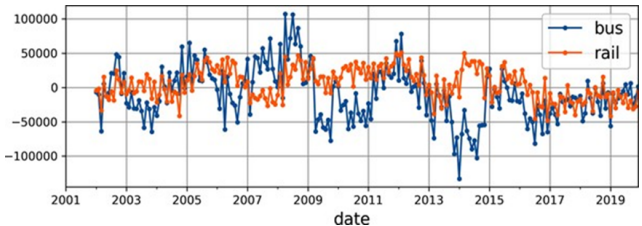
- Mở rộng ra 15 năm (2001-2015), **Hình 15-8** cho thấy rõ ràng các ngày lễ hội khiến lượng khách sụt giảm lớn hàng năm, và tổng thể hành khách biến động vĩ mô (Macro Trend).
- Để RNN mô hình hóa được điều này, nó cần "Trí nhớ Dài Hạn"vươn xa hàng trăm bước, cực kỳ khó nếu dùng `SimpleRNN`.



Hình 15-8: Tính thời vụ hằng năm và xu hướng dài hạn

Kỹ thuật Tiền Xử Lý: Sai phân Dài hạn

- Áp dụng sai phân 12 tháng liên tục.
- **Hình 15-9** cho thấy chuỗi kết quả cân bằng hơn quanh mốc 0.
- Mạng RNN hiện đại có thể tự phát hiện các trend này, nhưng trong thực tế, đưa vào chuỗi sai phân thường hội tụ nhanh và dự đoán chính xác hơn dữ liệu gốc hỗn độn.



Hình 15-9: Sự sai phân 12 tháng (Dài hạn)

Code: API Huấn luyện RNN cơ bản với Keras

Để xây dựng `SimpleRNN` trên Keras, dùng lớp mô phỏng nơ-ron hồi quy:

```
import tensorflow as tf
from tensorflow import keras

# Tao mang RNN 1 lop, 1 hidden unit
model = keras.Sequential([
    keras.layers.SimpleRNN(1, input_shape=[None, 1])
])

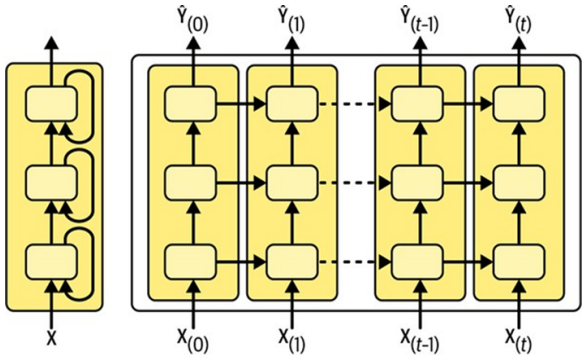
model.compile(loss="mse", optimizer="adam")
history = model.fit(X_train, y_train, epochs=20,
                    validation_data=(X_valid, y_valid))
```

Phân tích Code Keras RNN

- Tham số 1 trong `SimpleRNN(1)` là số lượng nơ-ron trạng thái ẩn (dimensionality of output space). Vì là dự đoán 1 bước tiếp theo (đại lượng vô hướng), nên chọn 1 nơ-ron.
- `input_shape=[None, 1]`:
  - `None`: Đại diện cho chiều dài của chuỗi (Time steps). Keras cho phép chuỗi dài ngắn tùy ý nếu ta đặt là None.
  - `1`: Số chiều (Features) tại mỗi bước thời gian. (Chỉ có 1 đặc trưng là lượng hành khách).
- Hàm Loss thường dùng cho bài toán dự báo chuỗi là Mean Squared Error (`mse`) hoặc Mean Absolute Error (`mae`).

Kiến trúc Mạng RNN Sâu

- Tương tự MLP Sâu, ta **xếp chồng (stacking)** các lớp RNN lên nhau tạo thành **Deep RNN**.
- Lớp dưới cùng nhận dữ liệu đầu vào. Đầu ra trạng thái ẩn $\mathbf{h}_{(t)}$ của lớp dưới làm đầu vào cho lớp trên tại **cùng thời điểm** *t*.
- **Hình 15-10** chỉ ra mũi tên lan truyền dọc theo cả 2 trục: Trục thời gian (ngang) và Trục không gian / Chiều sâu (dọc).



Hình 15-10: Mạng RNN sâu nhiều lớp

Code: Xây dựng Mạng RNN Sâu

- Yêu cầu quan trọng: Để lớp RNN dưới có thể đẩy chuỗi output lên lớp RNN trên, nó BẮT BUỘC phải bật `return_sequences=True`.

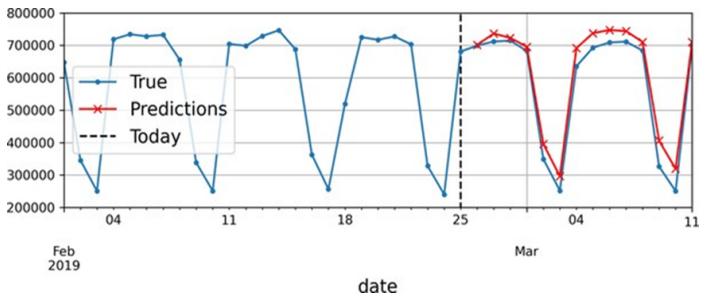
```
deep_model = keras.Sequential([
    # Lop RNN 1: Tra ve chuoi cho lop 2
    keras.layers.SimpleRNN(20, return_sequences=True, input_shape=[None, 1]),
    # Lop RNN 2: Tra ve chuoi cho lop cuoi
    keras.layers.SimpleRNN(20, return_sequences=True),
    # Lop RNN cuoi: Chi tra ve vector cuoi cung (Seq2Vec)
    keras.layers.SimpleRNN(1)
])
```

Dự báo nhiều bước (Multi-step Forecasting)

- Mạng trên chỉ dự báo $t+1$. Để dự báo xa (VD: $t+10$), có 2 cách:
  1. **Cách 1 (Dự đoán tuần tự - Autoregressive):** Dự đoán $t+1$, lấy giá trị đó gộp vào mảng đầu vào để đoán $t+2$, lặp lại 10 lần. (Dễ tích lũy sai số).
  2. **Cách 2 (Dự đoán đồng thời):** Đổi lớp cuối thành `Dense(10)` để bắt mạng xuất 10 giá trị cùng một lúc. Cách này thường ổn định hơn.

Kết quả dự báo đa bước của RNN Sâu

- **Hình 15-11** là kết quả dự báo một chuỗi nhân tạo nhiều bước đồng thời.
- Dấu $\times$ là dữ liệu đầu vào.
- Đường đứt nét là giá trị mục tiêu (Target).
- Dấu chấm tròn là giá trị dự đoán do mạng RNN sâu đưa ra.
- Mạng bám sát rất tốt đường đi tự nhiên của sóng dốc.



Hình 15-11: RNN Sâu dự báo đa bước khá chính xác

Vấn đề cốt lõi của SimpleRNN (Nhắc lại)

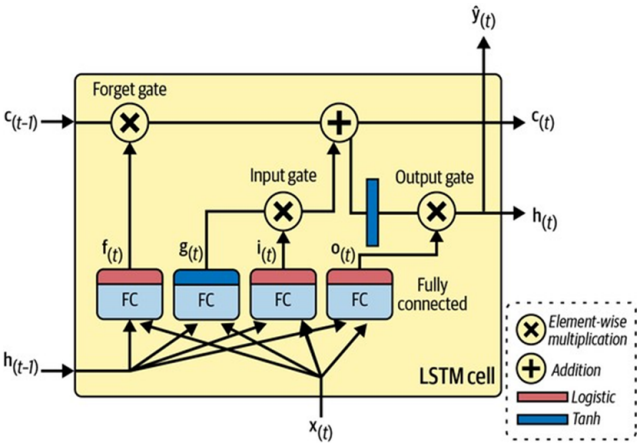
- SimpleRNN rất tồi trong việc nắm bắt **phụ thuộc xa (Long-Term Dependencies)**.
- Ví dụ câu NLP: *"Tôi sống ở Pháp từ nhỏ, sau 20 năm chuyển đến Mỹ, tôi vẫn nói trôi chảy tiếng [X]."*. Mạng cần nhớ chữ *"Pháp"* ở quá khứ rất xa để đoán chữ *"Pháp"*.
- Trọng số W nếu nhỏ thì Vanishing Gradient làm chữ "Pháp"bốc hơi sau vài bước thời gian.
- Để tạo ra "Trí nhớ Dài hạn", **Tế bào LSTM** được phát minh (Hochreiter và Schmidhuber - 1997).

Sơ lược về Tế bào LSTM (Long Short-Term Memory)

- Ý tưởng cốt lõi của LSTM là thay vì chỉ truyền một Trạng thái ẩn $h_{(t)}$, nó duy trì thêm một đường truyền phụ gọi là **Trạng thái tế bào (Cell State - $c_{(t)}$)**.
- $c_{(t)}$ giống như một đường ray cao tốc, chạy dọc xuyên suốt quá trình thời gian, cho phép thông tin truyền đi rất xa mà không suy giảm.
- Các cơ chế điều tiết (Gates - Cổng) kiểm soát việc ghi, đọc, xóa dữ liệu trên đường ray $c_{(t)}$ này.

Cấu trúc Tế bào LSTM chi tiết

- **Hình 15-12** là mô hình LSTM chuẩn. Đường ray ngang ở trên cùng là Trạng thái tế bào $c_{(t)}$. Đường ray dưới là Trạng thái ẩn $h_{(t)}$.
- Các phép toán tròn như $\otimes, \oplus$ là phép nhân phần tử và phép cộng vector.
- Cổng (Gate) sử dụng hàm kích hoạt Sigmoid (σ) (xuất giá trị [0, 1]) hoạt động như một cái "van": 0 là đóng hoàn toàn, 1 là mở hoàn toàn.



Hình 15-12: Cấu tạo bên trong Tế bào LSTM